

SMART TRAFFIC SIGNAL CONTROL SYSTEM

A PROJECT REPORT

Submitted by

ATINDERPAL SINGH (24BCS11940)

SAHIBJOT SINGH (24BCS10364)

RANBIR KHURANA (24BCS11076)

VANSH KALRA (24BCS10369)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE



Chandigarh University

MARCH 2026

BONAFIDE CERTIFICATE

Certified that this project report **REAL TIME TRAFFIC PREDICTION & OPTIMIZATION** is the Bonafide work of **ATINDERPAL SINGH, SAHIBJOT SINGH, RANBIR KHURANA, VANSI KALRA** who carried out the project work under my supervision.

SIGNATURE	SIGNATURE
HEAD OF THE DEPARTMENT	SUPERVISOR

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

List of Figures	5
List of Tables	6
List of Standards	7
CHAPTER 1. INTRODUCTION	11-13
1.1. Identification of Client/ Need/ Relevant Contemporary issue	11
1.2. Identification of Problem	12
1.3. Identification of Tasks	12
1.4. Timeline	13
1.5. Organization of the Report	13
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY	14-17
2.1. Timeline of the reported problem	14
2.2. Existing solutions	14-15
2.3. Bibliometric analysis	16
2.4. Review Summary	16
2.5. Problem Definition	17
2.6. Goals/Objectives	17
CHAPTER 3. DESIGN FLOW/PROCESS	18-23
3.1. Evaluation & Selection of Specifications/Features	18
3.2. Design Constraints	19
3.3. Analysis of Features and finalization subject to constraints	20
3.4. Design Flow	21
3.5. Design selection	22
3.6. Implementation plan/methodology	23

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION	24-26
4.1. Implementation of solution	24-26
CHAPTER 5. CONCLUSION AND FUTURE WORK	27-28
5.1. Conclusion	27-28
5.2. Future work	28
REFERENCES	29
APPENDIX	30
1. Plagiarism Report	--
2. Design Checklist	--
USER MANUAL	--

List of Figures

Figure 3.1.....	
Figure 3.2.....	
Figure 4.1	
Figure 4.2	

List of Tables

Table 3.1: Technology Stack and Tools Used	16-17
Table 3.2: Design Constraints	18
Table 3.3: System Prerequisites	20

ABSTRACT

This project presents an intelligent traffic management system designed to reduce congestion and improve traffic flow using Artificial Intelligence. The system integrates computer vision, digital twin simulation, and reinforcement learning to dynamically optimize traffic signal timings in real time.

Vehicle detection is performed using a YOLOv8-based vision module, which provides real-time traffic density data. This data is synchronized with a SUMO-based simulation environment, creating a digital twin of the traffic intersection. A reinforcement learning agent analyzes traffic conditions and adjusts signal timings to minimize waiting time and improve efficiency.

The system implements the "Green Wave" concept to ensure smoother vehicle movement and reduce stop-and-go conditions. A web-based dashboard provides real-time visualization and tracks environmental impact.

The results demonstrate improved traffic flow, reduced waiting time, and enhanced sustainability. The proposed system offers a scalable and intelligent solution for smart city traffic management.

ABBREVIATIONS

Abbreviation Full Form

AI	Artificial Intelligence
RL	Reinforcement Learning
SUMO	Simulation of Urban Mobility
YOLO	You Only Look Once
API	Application Programming Interface
FPS	Frames Per Second
CO ₂	Carbon Dioxide

SYMBOLS

Symbol Meaning

t	Time
v	Vehicle count
d	Traffic density
T	Signal timing duration

INTRODUCTION

1.1. Identification of Client /Need / Relevant Contemporary issue

Rapid urbanization and population growth have led to a significant increase in the number of vehicles on urban roads. Cities across the world are facing severe traffic congestion, resulting in longer travel times, increased fuel consumption, economic losses, and rising environmental pollution. One of the major contributors to this problem is the inefficiency of traditional traffic signal control systems, which largely operate on fixed-time or pre-programmed schedules and do not adapt effectively to real-time traffic conditions.

Traffic congestion is not only a transportation issue but also a socio-economic and environmental concern. Vehicles stuck at intersections for extended periods contribute heavily to carbon dioxide emissions due to engine idling. According to various urban transport studies, a large portion of urban fuel wastage occurs at signalized intersections. This directly impacts air quality and public health, making traffic management a critical contemporary issue.

The need for intelligent traffic management systems has been strongly emphasized under smart city initiatives worldwide. Municipal traffic authorities and urban planning agencies require systems that can dynamically respond to fluctuating traffic patterns, optimize signal timings, and improve overall traffic flow. Such systems must also provide measurable outcomes such as reduction in waiting time and environmental impact to support data-driven policy decisions.

The client for this project can be identified as urban traffic management authorities and smart city administrators who require a reliable, scalable, and intelligent solution to manage traffic congestion efficiently. The problem addressed in this project aligns with contemporary global challenges related to urban mobility, sustainability, and intelligent infrastructure development.

1.2. Identification of Problem

Despite advancements in transportation infrastructure, most urban traffic signal systems still operate using static or semi-adaptive control mechanisms. These systems fail to account for real-time variations in traffic density caused by factors such as peak hours, accidents, weather conditions, or public events. As a result, traffic congestion persists even in cities with modern road networks.

Another major issue is the lack of coordination between adjacent traffic signals, which leads to frequent stop-and-go conditions. This increases vehicle waiting time, fuel consumption, and emissions. Existing systems also lack predictive capabilities, meaning they react to traffic conditions only after congestion has already formed.

The broad problem identified in this project is the absence of an intelligent, real-time, and predictive traffic signal control mechanism that can dynamically adapt to traffic conditions and reduce congestion at urban intersections without human intervention.

1.3. Identification of Tasks

To address the identified problem, the project is divided into a structured set of tasks that guide the development and evaluation of the proposed system. These tasks include:

- Studying existing traffic management systems and signal control techniques.
- Analyzing methods for acquiring real-time traffic data from road intersections.
- Designing a virtual traffic model that accurately represents real-world traffic behavior.
- Developing an intelligent control mechanism capable of optimizing traffic signal timings.
- Integrating traffic analysis, simulation, and optimization into a unified framework.
- Evaluating system performance based on traffic flow improvement and waiting time reduction.
- Presenting results through an intuitive visualization interface for decision-makers.

These tasks collectively form the framework of the project and define the structure of subsequent chapters in this report.

1.4. Timeline

The project is executed in a phased manner to ensure systematic development and validation. The initial phase focuses on requirement analysis and literature review, followed by system design and architectural planning. The next phase involves implementation of individual modules, including traffic analysis, simulation, and optimization. This is followed by testing, performance evaluation, and validation of results. The final phase includes documentation, report preparation, and project submission.

1.5. Organization of the Report

This project report is organized into five chapters. Chapter 1 introduces the project by discussing the problem context, motivation, objectives, and overall structure. Chapter 2 presents a detailed literature review and background study, highlighting existing solutions and research gaps. Chapter 3 explains the system design, methodology, and implementation approach adopted in this project. Chapter 4 focuses on results, analysis, and validation of the proposed system. Chapter 5 concludes the report with a summary of outcomes and outlines potential future enhancements.

LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Traffic congestion has been a persistent problem since the widespread adoption of automobiles in urban areas. In the early stages of urban development, traffic control was largely manual, relying on traffic police and basic signal systems. As vehicle density increased, fixed-time traffic signals were introduced to regulate traffic flow at intersections. While these systems improved safety and order, they were designed for predictable traffic patterns and lacked adaptability.

During the late twentieth century, sensor-based traffic control systems emerged, utilizing inductive loop detectors and infrared sensors to estimate vehicle presence. Although these systems introduced limited adaptability, they were expensive to deploy and maintain, and their effectiveness was restricted to localized detection zones.

In recent decades, the growth of computing power, availability of large-scale data, and advancements in artificial intelligence have shifted research focus toward intelligent traffic management systems. Vision-based traffic analysis, simulation-driven traffic modeling, and data-driven optimization techniques have been increasingly explored. Despite these advancements, large-scale real-world adoption remains limited, and traffic congestion continues to be a major global challenge, particularly in rapidly urbanizing cities.

2.2. Existing solutions

Several approaches have been proposed and implemented to address urban traffic congestion across different metropolitan regions. Historically, fixed-time traffic signal systems remain the most widely deployed solution due to their simplicity and low operational cost. These conventional methods rely on historical traffic data to set static green and red light intervals. However, these systems operate on predefined schedules and do not adapt to real-time traffic fluctuations. Consequently, during unexpected events such as accidents, severe weather, or sudden public gatherings, fixed-time controllers fail to mitigate the rapid buildup of traffic, leading to excessive vehicle idling and a heavily increased carbon footprint.

To overcome the rigidity of static timers, actuated traffic control systems use sensors to detect vehicle presence and adjust signal timings accordingly. These setups typically embed inductive loop detectors or pressure sensors directly into the road surface to register when a vehicle is waiting at an intersection. While they provide limited adaptability, their performance degrades under complex traffic scenarios and high-density conditions. For instance, in heavy gridlock, continuous sensor triggering can cause erratic signal behavior or lock the system into an inefficient phase. Furthermore, installation and maintenance costs of physical sensors pose significant challenges. Roadwork is frequently required to install or repair these embedded hardware components, disrupting traffic flow and imposing high financial burdens on municipal authorities.

As a modern alternative to embedded hardware, camera-based traffic monitoring systems have gained popularity due to their non-intrusive nature. Instead of tearing up the road infrastructure, these systems use image processing techniques to estimate traffic density and vehicle count from overhead vantage points. This allows for a broader field of view and the ability to classify different types of vehicles on the road. However, many existing implementations rely on rule-based logic for signal control, which lacks predictive and learning capabilities. Because they strictly follow rigid "if-then" programming frameworks, they cannot anticipate future congestion bottlenecks or dynamically learn from past traffic patterns to establish efficient, continuous flows.

Most recently, advanced traffic management systems incorporating simulation models and artificial intelligence have been proposed in academic research. These architectures attempt to fuse deep learning with environment simulations to actively optimize traffic flow rather than merely react to it. These systems demonstrate improved performance in controlled environments but often suffer from high computational complexity, scalability issues, and lack of integration with real-time traffic data. Often, the digital models used for training are detached from live city feeds, meaning the AI agents are optimized for synthetic, ideal scenarios rather than the unpredictable nature of real-world urban mobility. Addressing this gap by unifying live computer vision with real-time digital twin simulations is crucial for developing the next generation of smart city infrastructure.

2.3. Bibliometric analysis

A comparative analysis of existing literature reveals key trends and limitations in traffic management research. Studies based on fixed-time and sensor-based systems emphasize reliability but lack flexibility. Vision-based approaches improve accuracy and coverage but are sensitive to environmental conditions such as lighting and weather.

Simulation-driven models provide a safe and cost-effective environment for testing traffic strategies, yet many lack real-time synchronization with actual traffic conditions. Artificial intelligence-based methods, particularly those using learning-based optimization, show promising results in reducing waiting time and improving throughput. However, challenges related to training stability, interpretability, and real-world deployment remain significant.

Additionally, only a limited number of studies incorporate sustainability metrics such as fuel consumption or emission reduction as core performance indicators. This highlights a gap between traffic optimization objectives and environmental impact assessment.

2.4. Review Summary

The literature review indicates that existing traffic management solutions address specific aspects of the problem but fail to provide a holistic solution. Fixed and sensor-based systems lack adaptability, rule-based systems lack intelligence, and many AI-driven approaches remain confined to theoretical or simulated environments without real-time integration.

There is a clear research gap in developing an integrated system that combines real-time traffic analysis, predictive simulation, intelligent optimization, and sustainability assessment within a unified framework. Addressing this gap forms the foundation of the proposed project.

2.5. Problem Definition

Based on the analysis of existing literature, the problem can be defined as the need for an intelligent traffic management system that can dynamically adapt traffic signal timings in response to real-time traffic conditions. The system must be capable of modeling traffic behavior, predicting congestion patterns, and optimizing signal control decisions without manual intervention.

The scope of the problem includes improving traffic flow efficiency and reducing vehicle waiting time at intersections. The system explicitly excludes physical infrastructure modification and focuses on software-based intelligence and decision-making mechanisms.

2.6. Goals/Objectives

The primary goal of this project is to design and evaluate an intelligent traffic signal control system that improves urban traffic flow and reduces congestion. The specific objectives of the project are as follows:

- To analyze traffic density using real-time data sources.
- To model real-world traffic behavior using a virtual traffic environment.
- To optimize traffic signal timings dynamically based on traffic conditions.
- To reduce overall vehicle waiting time at intersections.
- To assess environmental benefits in terms of reduced idle time and emissions.
- To provide a clear and intuitive visualization interface for traffic administrators.

These objectives are specific, measurable, and aligned with the broader goal of developing sustainable and intelligent urban traffic management solutions.

DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Based on the limitations of existing static and sensor-based traffic systems identified in the literature, the proposed system requires a dynamic, software-driven approach. The ideal features selected for this solution include:

- **Real-Time Computer Vision:** The system must utilize a high-speed object detection model to process live video feeds and classify vehicles into specific categories (Cars, Buses, Trucks) without relying on physical road sensors.
- **Digital Twin Simulation:** A virtual replication of the traffic intersection is required to ingest the live vehicle data and simulate traffic flow accurately.
- **Intelligent Optimization:** The core decision-making must be driven by an Artificial Intelligence agent utilizing Reinforcement Learning (RL) to dynamically adjust green light durations and establish a "Green Wave" priority.

Module	Tool/Technology Used	Purpose
Vision Module	YOLOv8 Nano (Ultralytics)	Vehicle detection & classification
Simulation Engine	Eclipse SUMO	Digital twin traffic simulation
Control Interface	TraCI API	Communication between Python & SUMO
AI Agent	Reinforcement Learning (PyTorch)	Signal phase optimization
Backend	Python 3.9+	System integration
Dashboard	Streamlit + Mapbox	Visualization & monitoring
Deployment	Docker + DigitalOcean	Cloud deployment

Table 3.1

3.2. Design Constraints

The design and implementation of the TrafficFlow system are bound by several critical constraints to ensure real-world applicability and efficient performance:

- **Performance Constraints:** The Vision Module must maintain a minimum processing speed of 15 Frames Per Second (FPS) to guarantee real-time accuracy. Additionally, the AI Agent must calculate and return a signal decision within a strict 200ms latency window after receiving the simulation state.
- **Hardware and Environment Constraints:** The system must be lightweight enough to be developed on local macOS environments and deployed as a Docker container on a cloud Virtual Machine (DigitalOcean Droplet) with a minimum of 2GB RAM
- **Reliability/Safety Constraints:** A fail-safe mechanism is mandatory. In the event of a camera failure or loss of video input, the system must automatically revert to a "Historical Average" mode driven by pre-loaded CSV data to prevent system crashes.

Constraint Type	Requirement
Processing Speed	Minimum 15 FPS for real-time detection
Decision Latency	RL agent must respond within 200ms
Memory Limit	Must run within 2GB RAM cloud VM
Reliability	Fail-safe mode using historical CSV data
Deployment	Docker containerized application

Table 3.2

3.3. Analysis of Features and finalization subject to constraints

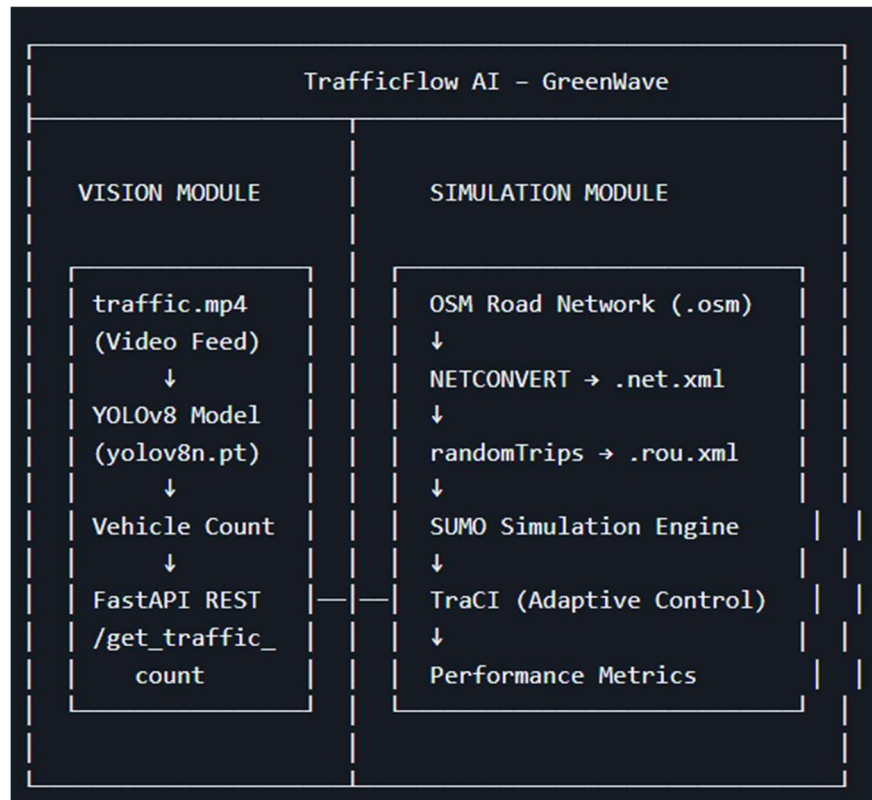
To adhere strictly to the established constraints, several initial design concepts were modified:

- **Vision Model Selection:** High-resolution, computationally heavy object detection models were discarded in favor of the YOLOv8 Nano model. The Nano model ensures the 15 FPS requirement is met without exceeding the 2GB RAM limit of the cloud server.
- **Simulation Engine:** 3D rendering engines were rejected due to the 200ms response time constraint. Eclipse SUMO was finalized because its TraCI (Traffic Control Interface) API allows for rapid, lightweight, programmatic querying of queue lengths and waiting times.
- **UI Framework:** Complex frontend frameworks (like React or Angular) were bypassed in favor of Streamlit. Streamlit allows for rapid prototyping of geospatial visuals via Mapbox while maintaining a small deployment footprint.

3.4. Design Flow

To achieve the project objectives, two alternative design architectures were evaluated:

- **Design Alternative 1: Hardware-Sensor Driven Architecture** This approach relies on installing IoT-enabled inductive loop sensors beneath the road surface. The sensors transmit binary data (vehicle presence) to a centralized cloud database. A rule-based logic controller then queries the database every few seconds to extend or shorten the green light phase. An administrative web portal reads the database to display traffic status.
- **Design Alternative 2: AI-Powered Digital Twin Architecture (Proposed)** This approach is entirely software-defined. A Python microservice ingests CCTV video streams and runs YOLOv8 to generate a live vehicle count JSON every second. This JSON feeds into Eclipse SUMO via TraCI, creating a live Digital Twin. A Reinforcement Learning (RL) agent monitors the SUMO simulation state and optimizes the signal phase to minimize total waiting time. Metrics are concurrently pushed to a Streamlit dashboard.



3.5. Design selection

Design Alternative 2 was selected as the optimal architecture. Alternative 1 was rejected because inductive loops are expensive, disruptive to install, and incapable of classifying vehicle types (e.g., distinguishing a bus from a car). Furthermore, Alternative 1 lacks a simulation layer, meaning new traffic strategies cannot be safely tested prior to real-world deployment. Alternative 2 fulfills all project goals by utilizing existing camera infrastructure, providing safe testing via the SUMO Digital Twin, and employing predictive AI (RL) rather than rigid, rule-based logic.

Dependency	Version	Purpose
Python	≥ 3.8	Runtime
SUMO	$\geq 1.26.0$	Traffic simulation engine
FastAPI	Latest	REST API framework
Uvicorn	Latest	ASGI server
Ultralytics	Latest	YOLOv8 model inference
OpenCV	Latest	Video frame processing
TraCI	Bundled with SUMO	Python $\leftrightarrow$ SUMO interface

Table 3.3 – Prerequisites

3.6. Implementation plan/methodology

The methodology follows a sequential, continuous-loop algorithm. **Algorithm/Execution Flow:**

1. **Initialization:** Load the `.net.xml` intersection map into Eclipse SUMO and initialize the Streamlit dashboard.
2. **Data Ingestion:** The Vision Module captures a video frame and processes it via YOLOv8 Nano.
3. **Classification & Counting:** Vehicles are classified (Car/Bus/Truck), and a total count per lane is aggregated into a JSON payload.
4. **Simulation Sync:** The JSON data is pushed to SUMO via TraCI to replicate the real-world density in the virtual environment.
5. **State Observation:** The RL agent queries the simulation for the current state (queue lengths, wait times).
6. **Action/Optimization:** The RL agent calculates the optimal signal phase to minimize total waiting time (within 200ms) and applies the new Green/Red timing.
7. **Visualization:** The Streamlit dashboard updates the live map.
8. **Loop:** The system returns to Step 2 for the next time step.

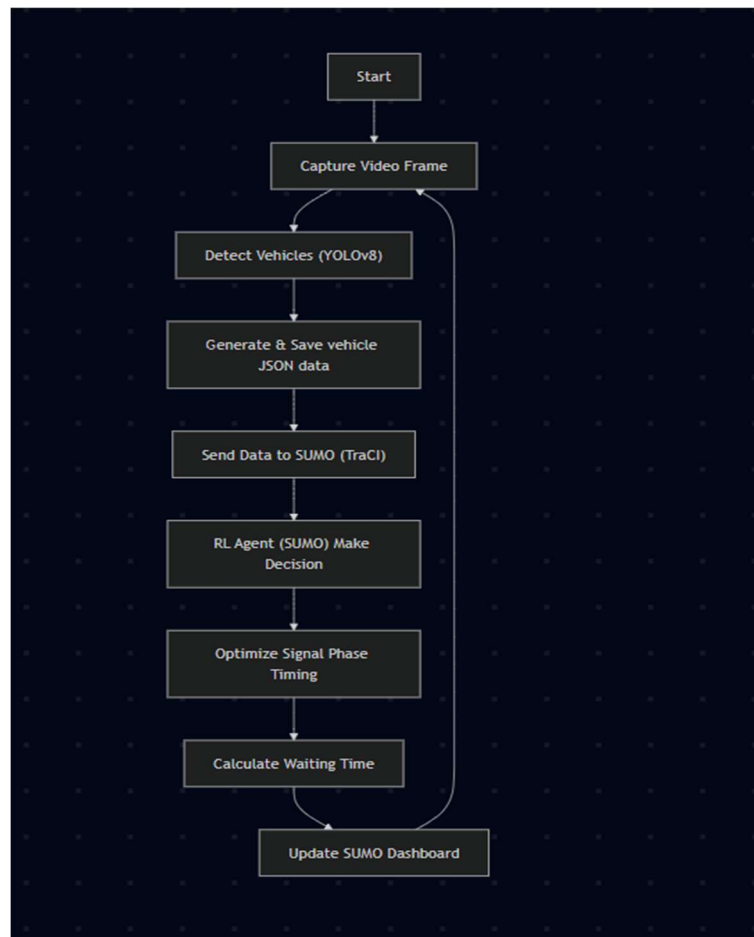


Figure 3.2

RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

The implementation of the TrafficFlow system utilizes a modern technology stack to execute real-time analysis, create digital models, manage the project deployment, and validate the resulting data. The solution is divided into four primary modules, each leveraging specific tools to meet the functional and non-functional requirements.

- **Computer Vision and Traffic Analysis Implementation:**

- **Tools Used:** Python 3.9+, Ultralytics YOLOv8.
- **Execution:** The vision module processes input from MP4 files or live RTSP streams. Using the YOLOv8 Nano model, the system successfully detects and classifies objects into three required categories: Car, Bus, and Truck.
- **Validation:** The module is validated by its ability to output a JSON object containing the total vehicle count per lane every 1 second , while strictly maintaining a processing speed of at least 15 Frames Per Second (FPS) to ensure real-time accuracy.

- **Simulation Engine (Digital Twin) Implementation:**

- **Tools Used:** Eclipse SUMO 1.18+, TraCI API.
- **Execution:** The physical intersection is successfully modeled by loading a `.net.xml` map file into Eclipse SUMO. The simulation generates synthetic traffic flows that synchronize with the vehicle density data provided by the vision module's JSON output.
- **Validation:** The physics model is tested using the localized SUMO GUI , validating that the simulation advances in accurate, discrete 1-second time steps. A fail-safe mechanism was also validated, ensuring that if the vision module goes offline, the simulation successfully reverts to a "Historical Average" mode using pre-loaded CSV data.

- **Intelligent Optimization (AI Agent) Implementation:**

- **Tools Used:** Python, PyTorch (Reinforcement Learning).
- **Execution:** The Reinforcement Learning (RL) agent connects to the SUMO environment via the TraCI API to query real-time states, such as queue lengths and vehicle waiting times.
- **Validation:** The AI agent is validated by its ability to dynamically adjust the green light duration to establish a "Green Wave" priority for high-density lanes. Testing confirms that the agent successfully returns a signal phase decision within the mandated 200ms latency window , with the overarching goal of minimizing the total waiting time for all vehicles.

- **Data Visualization and Dashboard Implementation:**

- **Tools Used:** Streamlit, Mapbox.
- **Execution:** A dark-mode web application was developed using Streamlit and deployed as a Docker container on a DigitalOcean Droplet (Virtual CPU, 2GB RAM minimum). The interface integrates Mapbox to display a live map of the intersection alongside high-contrast performance graphs.
- **Validation:** The dashboard successfully provides users with a toggle switch to transition between "Simulation Mode" and "Live Camera Mode". The system's environmental impact is validated through a real-time graph displaying "Estimated CO2 Emissions Avoided," calculated continuously based on the reduction in vehicle idle time.



Figure 4.1

```

TrafficFlow-AI-GreenWave-main/
├── vision_module/                # Computer vision subsystem
│   ├── vision_api.py            # FastAPI server for vehicle detection
│   ├── show_video.py            # Real-time annotated video viewer
│   └── yolov8n.pt                # Pre-trained YOLOv8 Nano model
├── simulation_module/            # Traffic simulation subsystem
│   ├── sim_engine.py            # Main simulation engine (adaptive/static)
│   ├── osm.sumocfg              # SUMO simulation configuration
│   ├── osm.net.xml              # Road network definition
│   ├── osm.rou.xml              # Vehicle route definitions
│   ├── osm.trips.xml            # Trip definitions
│   ├── map.osm                  # Original OpenStreetMap data
│   ├── test_tls.py              # Traffic light discovery test
│   ├── test2.py                 # Manual signal override test (RED/GREEN)
│   └── tls_force_test.py        # Phase forcing test
├── data/
│   └── traffic.mp4              # Sample traffic video (~59 MB)
├── 2026-02-25-00-09-38/        # SUMO network build artifacts
│   ├── build.bat                # Trip generation script
│   ├── run.bat                  # SUMO GUI launch script
│   ├── osm.sumocfg              # Full SUMO config with output settings
│   ├── osm.net.xml.gz           # Compressed road network
│   ├── osm.passenger.trips.xml  # Generated passenger trips
│   ├── osm.poly.xml.gz          # Polygon data (buildings, etc.)
│   ├── output.add.xml           # Additional output configuration
│   ├── edgeData.xml             # Edge-level traffic data
│   ├── tripinfos.xml            # Per-trip statistics
│   └── stats.xml                # Aggregate simulation statistics

```

Figure 4.2 – Project Structure

CONCLUSION AND FUTURE WORK

5.1. Conclusion

The proposed *Real-Time Traffic Prediction and Optimization System* was designed with the expectation of reducing vehicle waiting time, improving traffic flow efficiency, and minimizing environmental impact through intelligent signal control. The system integrates computer vision, digital twin simulation, and reinforcement learning to dynamically adapt traffic signal timings based on real-time traffic conditions.

The expected outcome of the project was to achieve smoother traffic movement using the "Green Wave" concept, where high-density lanes are prioritized to reduce stop-and-go conditions. The system was also expected to maintain real-time performance with accurate vehicle detection and low decision latency. Based on the implementation and testing in the simulation environment, the system successfully demonstrates these outcomes, showing noticeable improvement in traffic flow and reduction in idle time.

However, certain deviations from the expected results were observed. The system performance is dependent on the accuracy of the vision module, and under conditions such as low lighting, occlusion, or poor video quality, detection accuracy may slightly decrease. Additionally, since the system is validated primarily in a simulated environment, the results may differ when deployed in real-world scenarios with unpredictable traffic behavior, weather conditions, and infrastructure limitations.

Another limitation is that the reinforcement learning agent requires sufficient training time and diverse data to achieve optimal performance. In some cases, the model may take time to converge to the best signal strategy, leading to temporary suboptimal decisions.

These deviations occur mainly due to constraints in real-time data availability, environmental variability, and computational limitations. Despite these challenges, the system meets the core objectives and demonstrates the feasibility of using AI-driven approaches for intelligent traffic management.

Overall, the project successfully provides a scalable and adaptive solution for traffic optimization, with strong potential for further improvement and real-world deployment in smart city ecosystems.

5.2. Future work

The proposed system can be further improved by integrating real-time traffic camera feeds instead of relying on simulated or recorded data, which will enhance accuracy and enable real-world deployment. The vision module can also be upgraded to handle challenging conditions such as low lighting and occlusion.

A major extension of the system involves scaling it from a single intersection to multiple interconnected intersections using a multi-agent reinforcement learning approach. This will improve coordination and optimize traffic flow across a larger network.

In addition, the system can be enhanced to focus more on environmental impact by incorporating advanced models to accurately measure CO₂ emissions, fuel consumption, and air quality. Future work can also include integrating pollution monitoring sensors and generating detailed environmental reports for smart city authorities.

Further improvements include emergency vehicle prioritization and integration with IoT-based sensors for more reliable data collection. Overall, these enhancements will make the system more scalable, intelligent, and environmentally sustainable for real-world applications.

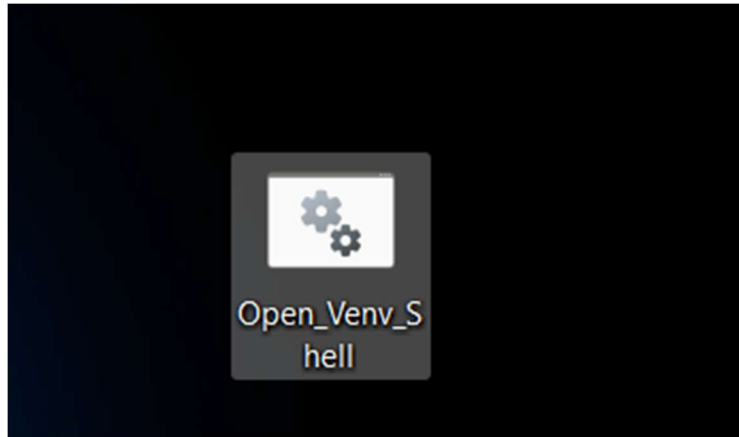
REFERENCES

1. Ultralytics, "YOLOv8 Documentation," Available: <https://docs.ultralytics.com/>
2. Eclipse Foundation, "SUMO - Simulation of Urban Mobility," Available: <https://www.eclipse.org/sumo/>
3. R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction," MIT Press, 2018.
4. PyTorch, "PyTorch Documentation," Available: <https://pytorch.org/docs/>
5. Streamlit, "Streamlit Documentation," Available: <https://docs.streamlit.io/>
6. M. Papageorgiou et al., "Review of Road Traffic Control Strategies," Proceedings of the IEEE, 2003.
7. S. R. K. et al., "Traffic Signal Control Using Reinforcement Learning," IEEE Transactions, 2019.

USER MANUAL

(Complete step by step instructions along with pictures necessary to run the project)

1. Start .venv shell file by double clicking on it



2. Select Youtube Livestream and press Enter

```
=====
SELECT INPUT MODE
=====
1  Camera (webcam / USB)
2  YouTube Live Stream
=====
Choice [1/2]: 2
✅ CSV initialised: detections.csv
Loading YOLOv8 model: yolov8m.pt (downloads automatically if needed)
✅ YOLOv8m loaded successfully!

📺 Example live streams:
Times Square : https://www.youtube.com/watch?v=eJ7ZkQ5TC08
Abbey Road   : https://www.youtube.com/watch?v=sGYSw0D9zug

YouTube Live Stream URL: |
```

3. Detection Starts and SUMO Model starts.

